

CertManager (Aplicativo N1) — Cambios para subir a producción

Documento: Checklist de cambios y configuración para producción. **Versión:** 1.0

El código viene listo para producción mediante el perfil `config.settings.prod`. NO hay que tocar código; todo se controla por **variables de entorno** (el `.env`). A continuación, lo que se debe completar / cambiar respecto a los valores por defecto.

1. Antes de instalar

#	Acción	Cómo
1	Solicitar la base de datos MySQL a Claro	Ver 02_necesidades_instalacion.md §3. Obtener host/usuario/clave/BD.
2	Solicitar el certificado TLS del FQDN del aplicativo	Clave + cadena.
3	Solicitar aperturas de firewall	Egress (§4) e ingress (§6) del doc 02.
4	Generar DJANGO_SECRET_KEY	<code>python -c "from django.core.management.utils import get_random_secret_key as g; print(g())"</code>

2. Completar el .env

Copiar `CLARO_NECESIDAD/.env.example` a `CLARO_NECESIDAD/.env` y completar. El `.env` **NO se versiona** (está en `.gitignore`): contiene secretos.

Cambios obligatorios respecto al ejemplo:

Variable	Default / placeholder	Cambiar a
DJANGO_SETTINGS_MODULE	<code>config.settings.prod</code>	(dejar)
DJANGO_SECRET_KEY	<GENERAR>	la clave generada (paso 1.4) — secreto
ALLOWED_HOSTS	<code>certmanager.claro.com.do</code>	el/los FQDN reales
CSRF_TRUSTED_ORIGINS	<code>https://certmanager.claro.com.do</code>	<code>https://<fqdn real></code>
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT	valores de ejemplo	credenciales reales de Claro (valores por separado) — <code>DB_PASSWORD</code> es secreto

Variable	Default / placeholder	Cambiar a
EMAIL_HOST / EMAIL_*	vacío	datos del SMTP corporativo (si se usa correo)
DEFAULT_FROM_EMAIL	certificados@claro.com.do	remitente real
LOG_DIR	/var/log/certmanager	(dejar, o ruta permitida)
CF_OWNER_PASSWORD	<DEFINIR>	contraseña del Owner (bootstrap) — secreto

3. Diferencias clave de producción (ya activas en `prod.py`)

Al usar `DJANGO_SETTINGS_MODULE=config.settings.prod`, automáticamente:

- `DEBUG = False`.
- **HTTPS forzado:** `SECURE_SSL_REDIRECT`, `HSTS` 1 año (`includeSubDomains`, `preload`), `cookies` `Secure` (sesión y CSRF), `X-Frame-Options: DENY`.
- Confía en el proxy TLS vía `X-Forwarded-Proto` (NGINX debe enviarlo — el script lo configura).
- Estáticos con hash + compresión (`ManifestStaticFilesStorage` + `WhiteNoise`).
- `SECRET_KEY` y `ALLOWED_HOSTS` **obligatorios** (la app no arranca sin ellos).
- MySQL con `utf8mb4` + `STRICT_TRANS_TABLES`.

Si el aplicativo va **detrás de un balanceador que ya hace TLS** y NGINX local recibe HTTP, mantener `SECURE_PROXY_SSL_HEADER` y asegurar el header `X-Forwarded-Proto: https`. Si NO hay TLS al frente (no recomendado), poner `SECURE_SSL_REDIRECT=0`.

4. Pasos de despliegue (los hace el script, aquí como referencia)

1. `pip install -r requirements/prod.txt` (en un virtualenv).
 - Si el índice privado de `obsforge` no está disponible: comentar esa línea o instalar `requirements/docker.txt` (sin `obsforge`). La app funciona igual.
2. Compilar CSS (`npm run build:css`) o copiar el `forge.css` ya compilado.
3. `python manage.py migrate` (crea/actualiza tablas en la MySQL de Claro).
4. `python manage.py collectstatic --no-input`.
5. (Opcional) `python manage.py compilemessages` (el `.mo` ya viene compilado).
6. **Owner + configuración + certificados:** `./data_update_certs_app.sh` (usa el `.env`; carga el Owner, la configuración por defecto y los certificados desde `./cert.txt`). Ver el README del repo.
7. Arrancar **dos servicios:** Gunicorn (web) y `run_scheduler` (tareas).
8. NGINX: reverse proxy 443 → `127.0.0.1:8000` + TLS.

5. Observabilidad (opcional)

- Por defecto los logs salen en **JSON** a `stdout` y a `/var/log/certmanager/ ( app.log , audit.log )`. Un agente (Promtail) los envía a Loki/Grafana.
- Para sintetizar con la librería corporativa **obsforge**: instalar el paquete (índice privado) y poner `OBSFORGE_ENABLED=1 + OBSFORGE_LOKI_PRESET=prod`. El preset `prod` **redacta PII** (correos/IP) en el stream; el dato real queda en `audit.log` y en la tabla `AuditLog`.

6. Hardening adicional recomendado (no bloqueante)

- **CSP en modo enforce** (`CSP_REPORT_ONLY=0`) tras validar que no rompe la UI.
- **2FA obligatorio** por organización (`require_2fa`) desde el panel de Seguridad.
- **Expiración de contraseñas** (panel de Seguridad) según política de Claro.
- Rotar `DJANGO_SECRET_KEY`, credenciales de BD/SMTP según política.
- Restringir el acceso a `/admin/` (ya limitado al superusuario de Django).

7. Migración del monitoreo (cert.txt)

Esta es la **migración de toda la data de monitoreo** al aplicativo (no es un paso opcional): el `cert.txt` trae los dominios, umbrales y correos de soporte. Solo está **pendiente del cert.txt actualizado**. Mientras no se tenga, se puede aprovisionar Owner + configuración con `--skip-certs` y completar la migración después. El `cert.txt` lo coloca el responsable en la raíz (no viene en el repo; está en `.gitignore`).

7.1 Colocar el archivo

Copiar el `cert.txt` en la **raíz del repo** (junto a `manage.py` y a `data_update_certs_app.sh`):

```
/opt/certmanager/cert.txt      # o la ruta donde quedó el aplicativo
```

7.2 Formato del archivo

Una línea por **destinatario**; el mismo dominio puede repetirse para varios correos. Campos separados por `|`:

```
dominio|correo|umbral_dias|puerto
```

Ejemplo:

```
sgv.claro.com.do|sp_canales_electronicos@claro.com.do|45|443
api.mi.claro.com.do|sp_canales_electronicos@claro.com.do|60|443
api.mi.claro.com.do|itredes@claro.com.do|60|443
ntpsfens.corp.codetel.com.do|itclienteservidornt@claro.com.do|50|443
```

7.3 Qué hace la carga (reglas ya implementadas)

- **Monitoreo**: todos los certificados quedan con alerta por **plataforma + correo**.

- **Ubicación:** dominios que empiezan con `ntp / ntt` → “**Servidor**”; los que contienen `claro.com.do` → “**netScaler**”; el resto vacío.
- **Grupos:** cada correo de soporte `sp*` se vuelve un grupo (los `- / _` se normalizan a uno solo); el cert se asigna a sus grupos `sp*`. El **Owner es ADMIN solo de `sp_canales_electronicos`**.
- **Destinatarios:** todos los correos del dominio (sp y no-sp) quedan como destinatarios de notificación.
- **Idempotente:** re-ejecutarlo no duplica nada.

7.4 Ejecutar

```
# El script carga CLARO_NECESIDAD/.env (Owner + SMTP) y lee ./cert.txt
sudo -u certmanager ./data_update_certs_app.sh

# Con Docker:
docker compose -f docker-compose.app.yml exec web ./data_update_certs_app.sh

# Validar primero sin escribir (opcional):
./data_update_certs_app.sh --dry-run
```

Variables que usa (del `.env`): `CF_OWNER_EMAIL`, `CF_OWNER_PASSWORD`, `CF_SMTP_*` (opcional). Para poblar el estado real de los certs al final, exportar `CF_RUN_CHECK=1` (lento: hace el handshake TLS a cada host).

7.5 Verificar la carga

```
sudo -u certmanager .venv/bin/python manage.py shell -c \
    "from apps.certificates.models import Certificate as C; print('certs:', C.objects.count())"
```

- En la UI: **Certificados** debe listar todo con su **Ubicación** y **Grupo**.

8. Verificación post-despliegue

```
curl -fsS https://<fqdn>/health/ # → {"status":"ok","database":true}
systemctl status certmanager certmanager-scheduler
journalctl -u certmanager -n 50
```

- Iniciar sesión con el Owner.
- Confirmar en **Configuración** → **Monitoreo** los valores (24h / timeout 10 / 1).
- Verificar que un certificado se chequea (“Probar ahora”).